



计算机系统（2）实践手册

作者：ACM 班课程助教

组织：ACM 班，上海交通大学

时间：2024-2025 春季学期

版本：1.1.1

目录

第 1 章 总述	1
1.1 作业内容	1
1.2 作业安排（初步）	1
第 2 章 项目环境准备	3
2.1 基准代码	3
2.2 建议的代码管理方式	3
2.3 在 QEMU 中启动 Linux 镜像	3
2.3.1 QEMU 安装（二选一）	3
2.3.1.1 软件包安装	3
2.3.1.2 源码编译安装	3
2.3.1.3 常见依赖问题	4
2.3.2 Linux 内核编译	4
2.3.3 BusyBox	4
2.3.3.1 直接安装	4
2.3.3.2 手动编译安装	4
2.3.4 安装 edk2	5
2.3.5 EDK2 编译（简述）	5
2.3.6 制作 initramfs	6
2.3.6.1 创建启动脚本	6
2.3.6.2 按需添加测试程序	6
2.3.6.3 打包文件系统	6
2.3.7 启动 QEMU	6
2.3.7.1 图形界面启动	6
2.3.7.2 字符界面启动	6
2.3.8 验证	7
2.4 Linux 环境部署 EDK2 开发环境	7
2.4.1 编译 EDK2	7
2.4.1.1 下载源码	7
2.4.1.2 网上找到的可能需要安装的一堆东西	7
2.4.1.3 make 一下 BaseTools	7
2.4.1.4 编译 UEFI 模拟器和 UEFI 工程	7
2.4.2 运行	8
2.4.2.1 只运行 UEFI 环境	8
2.4.2.2 运行 UEFI 环境和自定义 Linux	8
2.4.2.3 运行 UEFI 环境和 Ubuntu	9
第 3 章 系统启动	10
3.1 Read ACPI Table	10
3.1.1 ACPI Table 结构	11
3.1.2 .efi 编译与 EDK2 辅助文件撰写	11
3.2 Hack ACPI Table	12

3.3	UEFI 运行时服务	12
3.3.1	UEFI 启动阶段	12
第 4 章	特权态与系统调用	14
4.1	用户定义的系统调用	14
4.2	vDSO	14
4.3	无需中断的系统调用	15
第 5 章	内存管理	17
5.1	页表和文件页	17
5.2	内存文件系统	17
5.3	内存压力导向的内存管理	18
第 6 章	文件系统	19
6.1	inode 和扩展属性管理	19
6.2	用户态文件系统	19
6.3	用户空间下的内存磁盘	19
6.3.1	接口要求	19
6.3.2	测试	20
6.3.3	提示	20
第 7 章	网络与外部设备	21
7.1	tcpdump 和 socket 管理	21
7.1.1	功能目标	21
7.1.1.1	标准函数声明示例	21
7.1.2	tcpdump 定制化抓包	21
7.1.3	测试	22
7.2	高速通信库: NCCL	22
7.2.1	功能目标	22
7.2.1.1	标准函数声明示例	22
7.2.2	测试项与测试方式	22
7.2.3	更多细节	23
7.3	数据平面开发套件: DPDK	23
7.3.1	功能目标	23
7.3.1.1	标准函数声明示例	23
7.3.2	测试项与测试方式	23
7.3.2.1	测试项	23
7.3.2.2	测试方法	24
第 8 章	综合应用	25
8.1	使用 RDMA 的远程内存 Swap	25
8.1.1	指标	25
8.1.2	快速上手和实现重点提示	25
8.2	用户程序嵌入内核执行	26
8.2.1	指标	26
8.2.2	快速上手和实现重点提示	26
8.3	内核空间下的内存磁盘	27

8.3.1	指标	27
8.3.2	快速上手和实现重点提示	27
8.4	机密虚拟机 Trapless 的宿主协同内存管理	27
8.4.1	指标	27
8.4.2	快速上手和实现重点提示	28
参考文献		29
附录 A 细节说明		30
A.1	环境配置	30
A.1.1	EDK2	30
A.1.2	添加动态链接库	31

实践任务的背景

在 21 世纪 20 年代，机器学习的极具发展、大模型的快速迭代使得操作系统相关的内容成为“不受大家待见”的东西。学术界急速转向机器学习系统相关系统使得大家对于传统硬核的操作系统失去了兴趣。不仅班级内发生了转向，大多数实验室也在这一期间放弃原有传统系统研究而转向新兴与大模型结合的系统研究。为了适应这一潮流，对于对计算机系统没有兴趣的同学，助教们经过讨论认为需要将重点放在“如何使用好操作系统”和“如何理解操作系统的能力”两个部分。为此，助教们结合 2019、2020、2021、2022 级日常作业的内容，将小作业实践总结的目标进行整合，整理出了这一版实践的目标手册。

由于时间仓促，这一版本内容相对比较单薄。期待同学们在作业发展的过程中不断迭代增加内容。

第一版实践手册牵头讨论和实践的同学是郑文鑫（2017）、王鲲鹏（2022）、房诗涵（2022）、徐子绎（2022）、潘屹（2022）。

第 1 章 总述

1.1 作业内容

本系列作业旨在引导大家通过修改 Linux 内核的实现感受操作系统的功能。作业具体包括以下 6 个大类：启动、系统调用与特权态执行、内存管理、文件系统、网络与外部设备、安全。每个大类进一步根据难度和内容分为基础、实践、设计、综合。前三部分的内容如表 1.1 所示。综合部分的选题每一个选题横跨两个子项目，内容和目标如表 1.2 所示。

表 1.1: 分项内容

分项	基础	实践	设计
系统启动	Read ACPI Table	Hack ACPI Table	UEFI 运行时服务
系统调用 特权态	用户定义的系统调用	vDSO	无需中断的系统调用
内存管理	页表和文件页	内存文件系统	内存压力导向的内存管理
文件系统	inode 和扩展属性管理	FUSE	用户空间下的内存磁盘
网络 外部设备	tcpdump 和 socket 管理	NCCL	DPDK

表 1.2: 综合部分分项内容

项目	涉及分项
使用 RDMA 的远程内存 Swap	内存管理、网络与外部设备
用户程序嵌入内核执行	系统调用、内存管理
内核空间下的内存磁盘	内存管理、文件系统
机密虚拟机 Trapless 的宿主协同内存管理	内存管理、安全与虚拟化

1.2 作业安排（初步）

在本学期的作业中，我们将会定期安排大家进行习题课完成答疑和基本环境配置。小作业的初步计分规则如下（满分 100 分），正式发布在 3 月 1 日。

1. 基础部分：每个分项 6 分，共 30 分，必选；
2. 实践部分：每个分项 12 分，共 60 分；
3. 设计部分：每个分项 16 分，共 80 分；
4. 综合部分：每个分项 50 分，选一个共 50 分。

作业层级之间存在关联，即实践部分的内容可能会依赖基础部分的内容，设计部分的内容可能会依赖实践部分的内容，综合部分的内容可能会依赖设计部分的内容。因此，建议大家按照顺序完成作业。分数的设定同样存在层级关联，实践部分需要基础部分完全完成才可以计算，设计部分需要至少完成 2 个实践部分才可以计算，综合部分需要至少完成 1 个设计部分才可以计算。综合部分可能需要用到特别的硬件，如果需要使用特殊的硬件请联系助教。

诚信原则 如果出现以下情况，将会扣分：

1. 某一项作业与他人完全一致（即使是两人都是用大模型生成）：计小作业 0 分。
2. 某一项作业疑似使用大模型生成且没有对代码的合理解释：计该项作业 0 分。

作业提交 为了方便大家更加使用符合自己使用习惯的内核，我们允许使用任何 Linux 5.x.x 的内核版本完成作业。（不建议使用 6.0 以及更高版本，也不建议使用 5.0 之前的 4.x 版本）。请在 4 月 15 日前提交你的基础版本

号。对于每一个作业，你需要提交按照作业要求上载明的内容（部分的作业子项要求报告）和对应与 base 版本的 diff 文件。提交的文件格式为 pdf 和 patch 文件。所有作业的截止时间均为本学期 16 周周日晚上 23:59，但是自第 8 周开始，你可以提前提交作业（Code Review 仍然安排在 16 周），提前提交作业不会有额外的加分。

Code Review Code Review 将会在 16 周进行，具体时间和地点将会在之后公布。Code Review 的内容将会包括对你的代码的评价和对你的报告的评价。Code Review 的分数将会占到你分项的 30%。**你不应当认为 Code Review 是容易获得满分的。**

额外加分 我们鼓励大家通过向本 Repo 提交 PR 来提高你的分数。每一个 PR 将会有 0.1 至 2 分不等的加分（加小作业部分分），分数取决于表述的质量和作用。PR 的内容可以是对本 Repo 的文档的修正，对作业的补充，对作业的环境配置提供的建议等等。修正错别字、表述错误至多可以提交 10 次，每次 0.1 分。对作业描述的补充、环境配置按照书写的质量给分。其余类型的 PR 提交后请联系助教确认加分。要求释疑很大概率不会获得加分。重复提交不给分（重复修复同一个错别字、重复给出相似的作业描述补充等），按照 PR 的编号决定加分对象。加分给编号较小的人。最终加分不超过 20 分。

第2章 项目环境准备

2.1 基准代码

本系列的基准代码规则如下：

1. **Linux**: 任何体系结构下高于 5.10.20 版本的 Linux 内核都是可行的选项，你可以从<https://www.kernel.org/>中选择合适的版本。对于 `initramfs`，你可以选择任何一个发行版。如果你选择直接使用 `qcow` 或者其他完全预制好的 Ubuntu 镜像，你需要手动在内部更换内核以确保你的代码能够生效。
2. **QEMU**: 版本 ≥ 7 。
3. **EDK2**: 跟随<https://github.com/tianocore/edk2>选择合适的版本。其中 `Ubuntu_GCC5` 并不意味着 GCC 版本必须是 5。

2.2 建议的代码管理方式

建议单独开一个 `git` 本地仓库用于存放 Linux 内核，`init commit` 提交你下载的 Linux 内核代码。之后需要修改内核代码时，**开不同的分支**便于代码管理。最后提交作业时，可使用 `git diff` 命令生成 `patch` 文件

```
# 生成当前分支与主分支差异的补丁文件
git diff main > changes.patch

# 生成两个提交之间的差异
git diff <commit1> <commit2> > commit_diff.patch

# 生成两个分支之间的差异
git diff <branch1> <branch2> > branch_diff.patch
```

然后将生成的 `patch` 文件提交到作业中。

2.3 在 QEMU 中启动 Linux 镜像

注 简单起见你可以直接使用预制作好的 `qcow` Ubuntu 镜像作为磁盘镜像由 `qemu` 直接引导，具体教程：<https://documentation.ubuntu.com/public-images/en/latest/public-images-how-to/launch-qcow-with-qemu/>。如果你选用这个方案，你只需要确保 QEMU 正常安装即可。

注 本区域中 `.x.x` 部分请填入自己对应的版本号。

2.3.1 QEMU 安装（二选一）

2.3.1.1 软件包安装

```
sudo apt-get install qemu-system-x86
qemu-system-x86_64 --version
```

2.3.1.2 源码编译安装

```
# 安装依赖
sudo apt-get install git libglib2.0-dev libfdt-dev libpixman-1-dev zlib1g-dev ninja-build
```



```
# 下载编译
mkdir ~/kvm && cd ~/kvm
wget https://download.qemu.org/qemu-7.2.0.tar.xz
tar -xf qemu-7.2.0.tar.xz
cd qemu-7.2.0
./configure --target-list=x86_64-sofmmu
make -j$(nproc)
```

2.3.1.3 常见依赖问题

- **ERROR: Cannot find Ninja** → 执行 `sudo apt install ninja-build`
- **ERROR: glib-2.56 required** → 执行 `sudo apt install libglib2.0-dev`
- **Dependency "pixman-1" not found** → 执行 `sudo apt install libpixman-1-dev`

2.3.2 Linux 内核编译

```
# 安装依赖
sudo apt-get install git fakeroot build-essential ncurses-dev xz-utils libssl-dev bc flex libelf-dev bison
mkdir ~/kernel && cd ~/kernel
wget https://git.kernel.org/pub/scm/linux/kernel/git/stable/linux.git/snapshot/linux-5.x.x.tar.gz
tar -xf linux-5.x.x.tar.gz
cd linux-5.x.x

make defconfig # 使用默认配置
make -j$(nproc) # 开始编译
```

编译输出文件: `arch/x86/boot/bzImage`

在编译过程中，可能会发生编译错误的情况。这可能是 GCC 版本过高导致的。这里给出一种可行的组合：Linux-5.19.17, 使用 GCC-12 进行编译，操作如下：

```
sudo apt install gcc-12 g++-12 # 安装gcc-12
sudo update-alternatives --install /usr/bin/gcc gcc /usr/bin/gcc-12 70
sudo update-alternatives --install /usr/bin/g++ g++ /usr/bin/g++-12 70 # 切换 gcc 默认版本
```

经过这些操作后，应该可以正常编译 Linux 内核了。

2.3.3 BusyBox

2.3.3.1 直接安装

直接使用你的开发环境自带的包管理器安装 busybox，然后在后面制作 initramfs 的过程中添加这两步：先把 busybox 拷贝进 initramfs：

```
cp $(which busybox) "${INITRAMFS_DIR}/bin/busybox"
```

然后在系统启动脚本里添加：

```
/bin/busybox --install -s /bin
```

2.3.3.2 手动编译安装

这里可以采用其他发行版（例如 Ubuntu 发行版等）。

```
mkdir ~/kvm && cd ~/kvm
wget https://busybox.net/downloads/busybox-1.x.x.tar.bz2
tar -xf busybox-1.x.x.tar.bz2
cd busybox-1.x.x

# 配置命令
make menuconfig
# 选中: Settings > Build Options > [*] Build static binary
# 取消: Shells > [ ] Job control

make -j$(nproc) && make install
```

如果在编译过程中出现错误 (TCA_CBQ_MAX undeclared)，一种解决方式是直接删除出错的“tc.c”文件。之后就可以正常编译了 (版本 1.32.0 可以用此方法解决)。

2.3.4 安装 edk2

若不需要给 edk2 添加自定义功能，你可以直接使用包管理器安装预编译的 edk2。Ubuntu 24.04 下可以执行：

```
sudo apt-get install ovmf
```

安装完成后，你可以在 /usr/share/OVMF 目录下找到 OVMF_CODE_4M.fd 和 OVMF_VARS_4M.fd 两个文件，将它们的路径填入后文 qemu-system-x86_64 的启动参数中即可使用。

2.3.5 EDK2 编译 (简述)

EDK2 的可编译源码在 git submodule 里，因此相对简单可靠的方式就是直接按照官网所说的那样：

```
git clone -b stable/202408 https://github.com/tianocore/edk2.git
cd edk2
git submodule update --init
```

在编译之前，需要在系统里安装一些基础的包 (例如 base-devel、uuid、nasm、acpica)，然后就可以编译 BaseTools。

```
sudo apt install iasl nasm
```

在笔者实践的过程中，遇到了默认安装的 nasm 版本过低，导致后续编译出错的问题。读者可参考后文 2.4.1.2 的内容，自行编译 nasm。

```
source edksetup.sh
make -C BaseTools -j$(nproc)
```

然后编辑 Conf/target.txt (参考 <https://github.com/tianocore/tianocore.github.io/wiki/How-to-build-OVMF>):

```
ACTIVE_PLATFORM = OvmfPkg/OvmfPkgIa32X64.dsc
TARGET_ARCH      = IA32 X64
TOOL_CHAIN_TAG   = GCC5
```

最后，执行：

```
build
```

然后就可以在 Build/Ovmf3264/DEBUG_GCC5/FV/ 目录找到 OVMF_CODE.fd 和 OVMF_VARS.fd 两个编译出的 OVMF 文件。(这里的 Ovmf3264 和 DEBUG_GCC5 和编译选项有关，也是在 Conf/target.txt 里面配置。)

2.3.6 制作 initramfs

2.3.6.1 创建启动脚本

```
cd busybox-1.x.x/_install/
mkdir -p proc sys dev tmp
cat > init << 'EOF'
#!/bin/sh
mount -t proc none /proc
mount -t sysfs none /sys
mount -t tmpfs none /tmp
mount -t devtmpfs none /dev
echo "Hello Linux!"
sh
poweroff -f
EOF
chmod +x init
```

2.3.6.2 按需添加测试程序

initramfs 就是我们简易版 Linux 的文件系统，我们可以在里面添加测试程序。如果是用 C 语言编写，在编译时添加 `-static` 选项静态编译，然后添加到 `busybox-1.x.x/_install/bin` 里即可。如果程序难以静态编译，则可以使用附录中的脚本将程序所依赖的动态链接库添加到 initramfs 里。

2.3.6.3 打包文件系统

```
cd busybox-1.x.x/_install/
find . -print0 | cpio --null -ov --format=newc | gzip -9 > ../initramfs.cpio.gz
```

2.3.7 启动 QEMU

2.3.7.1 图形界面启动

```
qemu-system-x86_64 \
  -kernel ./kernel/linux-5.x.x/arch/x86/boot/bzImage \
  -initrd ./kvm/busybox-1.x.x/initramfs.cpio.gz \
  -append "init=/init"
```

2.3.7.2 字符界面启动

```
qemu-system-x86_64 \
  -kernel ./kernel/linux-5.x.x/arch/x86/boot/bzImage \
  -initrd ./kvm/busybox-1.x.x/initramfs.cpio.gz \
  -nographic \
  -append "init=/init console=ttyS0"
```

使用 EDK2 的 UEFI 模拟器启动时，需要使用 2.4.2 中的命令。

2.3.8 验证

- 检查内核版本: `uname -a`
- 查看启动参数: `cat /proc/cmdline`
- 测试基本命令: `ls, mount`

2.4 Linux 环境部署 EDK2 开发环境

2.4.1 编译 EDK2

python 的版本要求 ≥ 3.11 ,

2.4.1.1 下载源码

```
git clone https://github.com/tianocore/edk2.git
cd edk2
git submodule update --init #大概需要几分钟
cd ..
```

2.4.1.2 网上找到的可能需要安装的一堆东西

```
sudo apt-get install build-essential uuid-dev
sudo apt-get install uuid-dev nasm bison flex
sudo apt-get install libx11-dev x11proto-xext-dev libxext-dev
```

将 NASM 更新到 2.15.x 以上版本, 当前的 edk2 项目需要依赖 nasm2.15 以上版本, 否则编译会报错 nasm
各版本下载链接: <http://www.nasm.us/pub/nasm/releasebuilds/?C=M;O=D> (特别注意, 推荐下载.tar.xz 格式的 source code, .zip 解压后的文件似乎都是 CRLF 格式, 可能会导致编译错误)

```
cd nasm-2.16
./autogen.sh
./configure
make
make install
nasm --version
```

2.4.1.3 make 一下 BaseTools

然后 cd 到 edk2/BaseTools 下

```
make
```

显示 ok 则成功。注意这里所有的文件应该是 LF, 不能是 CRLF, 不然会出问题。可以先检查一下。

然后到 edk2 的 conf 目录下, 创建 target.txt(template 在 readme 里面) 然后修改: `TARGET_ARCH = X64`
`TOOL_CHAIN_TAG = GCC5`

2.4.1.4 编译 UEFI 模拟器和 UEFI 工程

先到 edk2 目录下, 然后执行

```
source edksetup.sh
build
```

然后到 target.txt 里面改一下: ACTIVE_PLATFORM = OvmfPkg/OvmfPkgX64.dsc

2.4.2 运行

2.4.2.1 只运行 UEFI 环境

在一开始的任务中,完全没有必要拉起一个完整的系统环境,甚至连 Linux 系统内核镜像都不需要启动,只需要全程在 UEFI 环境里执行。

此时的 UEFI 环境,简单来讲由两部分构成:

1. OVMF 文件: 用于在虚拟机中,提供 UEFI 固件,从而启动 UEFI 环境。
2. 若干 .efi 文件: UEFI 环境下的可执行文件

之前所说的那个 ‘build’ 指令,用处就是编译 OVMF 固件。你可能还希望使用这个命令编译一个更强大的 UEFI Shell

```
build -p ShellPkg/ShellPkg.dsc
```

然后你会找到一个 AcpiViewApp.efi 文件 (可以使用 `find . -name "AcpiViewApp.efi"` 来寻找它),把它挂载进文件系统,你就可以在 UEFI Shell 里使用它来执行 EDK2 预先写好的一个 AcpiView。详细的示例,参考[附录](#)

进入 UEFI Shell 后,执行下列命令:

```
fs0:          # go to `uefi` directory
AcpiViewApp.efi # run AcpiView
reset         # if you add `-no-reboot` to the qemu command, this will exit the UEFI Shell
```

注 在 UEFI 环境下,如果 `backspace` 键无法使用,可以使用 `Ctrl+H` 键来代替。

注 将 UEFI 指令写入 `uefi` 目录下的 `startup.nsh` 文件中,可以实现开机自动运行。

2.4.2.2 运行 UEFI 环境和自定义 Linux

首先将编译好的内核镜像和 `initramfs` 拷贝到 `uefi` 目录下 (也可以软链接)。然后执行:

```
cp /path/to/kernel/arch/x86/boot/bzImage uefi/bzImage
cp /path/to/initramfs.cpio.gz uefi/initramfs.cpio.gz
qemu-system-x86_64 \
  -drive if=pflash,format=raw,readonly=on,file={Your OVMF_CODE.fd here} \
  -drive if=pflash,format=raw,file={Your OVMF_VARS.fd here} \
  -drive file=fat:rw:./uefi,format=raw,if=ide,index=0 \
  -nographic
```

进入 UEFI 环境之后,执行下列命令:

```
fs0:          # go to `uefi` directory
# boot the kernel
bzImage initrd=initramfs.cpio.gz init=/init console=ttyS0
reset         # in case the kernel does not boot
```

启动 Linux 系统后,可以通过如下命令将 `uefi` 文件夹挂载到 `/mnt/efi` 目录:

```
mount /dev/sda1 /mnt/efi
```

若使用该方法运行虚拟机,在 Linux 系统使用 `poweroff -f` 重启虚拟机,会重新进入 UEFI Shell 环境。

2.4.2.3 运行 UEFI 环境和 Ubuntu

首先根据前文2.3 制作一个 Ubuntu 镜像 ubuntu.img。

然后运行 qemu：

```
qemu-system-x86_64 \  
-machine q35 \  
-m 8G \  
-smp 4 \  
-snapshot \  
-drive if=pflash,format=raw,unit=0,file={Your OVMF_CODE.fd here},readonly=on \  
-drive if=pflash,format=raw,unit=1,file={Your OVMF_VARS.fd here} \  
-drive file="/path/to/ubuntu.img",format=qcow2,if=virtio \  
-drive file=fat:ro: "/path/to/folder/uefi",format=raw,if=ide,index=1 \  
-nographic \  
-net none \  
-serial mon:stdio
```

注意到我们将一个文件夹 uefi 也挂载进 UEFI 环境中。它在 UEFI Shell 中的路径为 fs0:。我们可以在把自己编写的 UEFI 程序放在这里。

而 Ubuntu 启动文件夹在 UEFI 中的路径为 fs1:。进入这个目录后，有两种方法可以启动 Ubuntu：

```
/EFI/BOOT/BOOTX64.EFI  
# or  
/EFI/ubuntu/grubx64.efi
```

进入 Ubuntu 系统后，Ubuntu 的启动文件夹位于 /boot/efi 下。

注 上述 qemu 启动命令中，-snapshot 选项表示虚拟机关机后，对虚拟机状态的修改不会保存到磁盘上。该选项非必须。

第3章 系统启动

计算机上电后，系统会进行一系列初始化操作完成硬件设备的逐个启动。早年这一个过程由执行每个设备上的 ROM 中存储的代码（一般称为固件 firmware）结合 BIOS 完成。随后，更加通用的 UEFI 被提出，UEFI 的全称是 Unified Extensible Firmware Interface，它是一套统一使用 C 语言编写在 OS 启动前的固件代码段。

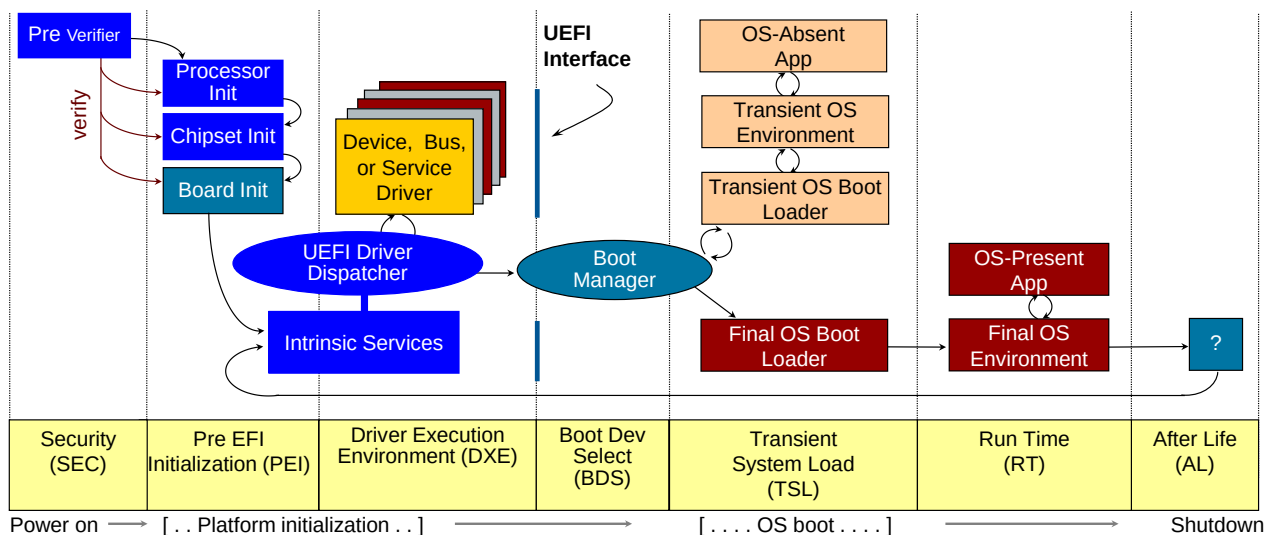


图 3.1: UEFI 启动过程

这些固件代码段还会收集硬件信息，并且移交给操作系统。早年的黑苹果、伪造 OEM 激活 Windows 7 都采用劫持 UEFI 启动后 ACPI Table 内容实现欺骗 OS 的效果。在这个专题的实践中，我们需要体验读取、修改系统启动过程中的各个表，并且尝试传递更多的信息给操作系统。

3.1 Read ACPI Table

目标：修改 EDK2 的 UEFI 组件，在其中增加函数：PrintAllACPITables。该函数不传入任何参数，要求打印每一张表的地址、长度以及每一张其中所有的信息与校验信息。

可以参考 EDK2 下的 AcpiView.c 中的 AcpiView 和 EfiLocateFirstAcpiTable 函数，但是不得直接复制。

此外，关于 ACPI 的更多信息，可以参考[这本手册](#)来学习，为了完成对应的任务，也许你只需要关注 ACPI 表的构成和 checksum 的含义即可。

具体测评指标：

- 输出所有 ACPI 表的完整地址映射；
- 每个表的输出必须包含：物理地址（64 位十六进制）、表长度（32 位十进制）、OEM ID（6 字节 ASCII）、校验和（8 位十六进制）。

为了能够在 UEFI Shell 中运行该函数，你可以学习如何自定义一个模块，或者在一个模块中添加应用。然后，你可以用这样的代码来测试你的程序：

```
cp edk2/Build/Shell/DEBUG_GCC5/X64/...{Your application efi file here} ./uefi/AcpiView.efi
qemu-system-x86_64 \
-drive if=pflash,format=raw,readonly=on,file={Your OVMF_CODE.fd here} \
-drive if=pflash,format=raw,file={Your OVMF_VARS.fd here} \
-drive file=fat:rw:./uefi,format=raw,if=ide,index=0 \
-nographic \
-no-reboot
```

为了自动运行你的程序，你可以在 `uefi` 文件夹中创建 `startup.nsh` 文件。该文件中的指令会在进入 UEFI Shell 时自动执行，参考内容如下：

```
fs0:      # change to the filesystem where the acpi-viewer.efi is located
AcpiView.efi  # run the AcpiView.efi
reset      # shut down the VM (because -no-reboot is set)
```

3.1.1 ACPI Table 结构

ACPI Table 是一个树状结构，其第一张表是 RSDP(Root System Description Pointer)，它里面存放了 RDST/XDST 的地址。RDST 是 32 位地址，XDST 是 64 位地址，其功能是一样的。而 RDST/XDST 中记录了其他表的地址。

注意在 FADT(Fixed ACPI Description Table) 中有还记录 FACS 和 DSDT 的地址，这个两个表无法直接在 RDST/XDST 中找到。

3.1.2 .efi 编译与 EDK2 辅助文件撰写

为了编译出能在 UEFI 中运行的 .efi 格式组件，我们需要在编译时指定特殊的编译目标。

例如，在 Rust 中，你只需要在 `rust-toolchain.toml` 中指定 `targets = ["x86_64-unknown-uefi"]`。

或者，使用 `gcc` 或 `clang`，也可以通过一系列复杂的命令行参数达成这一目的。

当然，为了利用 EDK2 中已有的结构体、宏、函数定义等，使用 EDK2 环境进行组件编写是比较好的选择。而且，也只需要 `source edksetup.sh`；`build -p MyACPIPkg/MyACPIPkg.dsc` 即可像其他 Pkg 一般完成编译。

唯一的障碍是，在 EDK2 中，除了 .c 代码以外，还需要撰写比较麻烦的 .dsc 与 .inf 文件。其他组件(如 `helloworld.c`)的这两个文件可以作为撰写的参考，但由于环境复杂，文件内容相对繁多。但实际上，只有少数 field 需要定义和填充。

下面是一个 Minimum Working Example:

In MyACPI.inf :

```
[Defines]
INF_VERSION      = 0x00011451      # 此处出于一些规定需要 0x00010005
BASE_NAME        = MyACPI
MODULE_TYPE      = UEFI_APPLICATION # 否则无法生成 .efi
VERSION_STRING   = 1.0
ENTRY_POINT      = MyACPIFunc      # 函数入口

[Sources]
MyACPI.c          # 源代码相对 .inf 的路径

[Packages]
MdePkg/MdePkg.dec
MdeModulePkg/MdeModulePkg.dec

[LibraryClasses]
UefiApplicationEntryPoint
UefiLib
PcdLib
```

In MyACPIPkg.dsc :

```
[Defines]
PLATFORM_NAME    = MyACPI
PLATFORM_GUID    = XXXXXXXX-XXXXXXX # 应由各位自行生成
PLATFORM_VERSION = 0.01
OUTPUT_DIRECTORY = Build/MyACPI    # .efi 输出目录
```

```

SUPPORTED_ARCHITECTURES = IA32|X64|EBC|ARM|AARCH64|RISCV64|LOONGARCH64
BUILD_TARGETS           = DEBUG|RELEASE

#include MdePkg/MdeLibs.dsc.inc

[LibraryClasses]
  UefiApplicationEntryPoint|MdePkg/Library/UefiApplicationEntryPoint/UefiApplicationEntryPoint.inf
  # ...
  # 此处取决于你 #include 的头文件等依赖库
  # 看 build 时报的缺文件错误信息即可定向修补
  # 比较取巧的方式是直接复制其他 .dsc 中的一坨

[Components]
  MyACPIPkg/MyACPI.inf          # .inf 相对 edk2/ 根目录的路径

```

3.2 Hack ACPI Table

目标：修改 EDK2 的 UEFI 组件，在其中增加函数：ChangeACPITable。该函数传入一个 UINTN 的对象表明修改第几张表、输入 EFI_ACPI_DESCRIPTION_HEADER* 表明待修改的数值，要求修改完成后打印每一张表的地址、长度以及每一张其中所有的信息与校验信息。

测试：通过引导 Linux 后读取对应的 ACPI 表检查输出进行验证。

具体评测指标：

- 修改 EFI_ACPI_DESCRIPTION_HEADER 中所有字段（包括 Signature 保留字段）且自动生成正确校验和的比例达到 100%；
- 系统重启 3 次后修改仍然有效；
- 内核日志（dmesg | grep ACPI）无 CHECKSUM_ERROR 警告或其他 ACPI 警告。

3.3 UEFI 运行时服务

目标：增加一个新的 UEFI 运行时服务，使得 Linux 系统可以调用这个新的服务导出硬件运行时信息。

提示：需要修改 UEFI 固件和 Linux 内核（sysfs 部分），方便直接读取导出的结果，你可以不用真的实现 UEFI Runtime Service，只需要向 OS 传递信息即可，可以通过 ACPI 表传递。

测试：展示

3.3.1 UEFI 启动阶段

UEFI 系统在启动过程中会经历三个不同的阶段。

平台初始化阶段（Platform Initialization）

这个早期启动阶段主要由 UEFI 固件完成。它在 UEFI 平台初始化规范（UEFI Platform Initialization Specification）中有描述，该规范与主 UEFI 规范是分开的。

启动服务阶段（Boot Services）

此阶段会加载 UEFI 启动驱动（UEFI Boot Service Drivers）、运行时驱动（UEFI Runtime Drivers）和应用程序（UEFI Applications）。可以同时使用 UEFI 的启动服务（Boot Services）和运行时服务（Runtime Services）。此阶段通常以运行加载操作系统的引导加载程序（bootloader）为结束。该阶段在调用 ExitBootServices() 后结束，系统进入运行时模式（Runtime mode）。

运行时阶段（Runtime）

运行操作系统（如 Linux 或 Windows）时，通常处于该阶段。在运行时模式下，UEFI 功能受到很大限制，不能再使用 UEFI 启动服务，但 UEFI 运行时服务仍可用。一旦系统进入运行时模式，只有系统重启后才能重新进入启动服务阶段。

进入运行时模式后，在启动服务阶段加载的 UEFI 运行时驱动的代码仍然会保留在内存中，其中的函数能被 UEFI 事件（UEFI Event）触发调用。

第 4 章 特权态与系统调用

操作系统对特权态的管理是操作系统的重要组成部分，影响了用户态程序的编写。这些系统调用也是操作系统向用户态提供抽象的重要部分，例如 Linux 中将设备的读写都转换为文件读写，因此对应的系统调用都和文件的操作读写相关。Windows 的系统调用接口则更为复杂一些，因为其内核的构成并不是单纯的宏内核，对于最小执行单元也有不同的定义。在这个专题的实践中，我们的目标是体验操作系统内核在通过系统调用进行系统管理上的重要作用。

4.1 用户定义的系统调用

目标：新增内核的键值存储模块（Key-Value Store），要求提供 read 接口和 write 接口。内核的键值数据需要存储在进程相关的结构体中，键值存储的对象是进程为对象。

提示：修改用户态系统调用的二进制文件和内核的系统调用接收器。

指标与评测要求

- 可评测指标：系统调用延迟（用户态到内核态切换时间 + 处理时间）、多线程下键值读写正确性（100 并发线程下无竞争条件）
- 吞吐要求：单核每秒可处理不少于 10,000 次空操作读写请求

实现规范

1. 接口定义：

```
// 用户态接口
int write_kv(int k, int v); // 返回写入字节数，错误返回-1
int read_kv(int k);        // 成功返回对应值，错误返回-1
```

2. 内核侧实现：

- 在进程控制块 (task_struct) 中添加 kv_store 字段：struct hlist_head kv_store[1024];
- 采用开放寻址哈希表实现，哈希函数：k % 1024
- 同步机制：每个哈希桶使用独立自旋锁 (spinlock_t)

测试方案

- 创建 100 线程交替随机读写，最终验证数据一致性
- 使用 sysbench 工具发起 100 万次随机请求
- 开启 KASAN 和 LOCKDEP 内核调试选项下测试并发正确性

4.2 vDSO

在上一个体验中，你已经增加了系统调用感受到了系统的服务。然而每次系统调用还是需要通过异常或者中断进行特权态切换，会带来较大切换开销。经过观察，可以发现一部分系统调用并不会泄露内核的状态也不会修改内核。对于这些系统调用可以在用户态以只读共享页面的方式直接进行而无需进入操作系统内核。Linux 中提供了 vDSO 支持这一想法。

目标：在 vDSO 中新增一个读取当前进程对应的 task_struct 中的所有信息的函数。

指标与评测要求

- 性能：读取数据延迟需小于 100ns（依据机器有所不同）
- 正确性：返回的 `task_struct` 指针地址与内核地址空间数据匹配

实现规范

1. 接口设计：

```
// vDSO 暴露接口
struct task_info {
    pid_t pid;
    void *task_struct_ptr;
    // 其他关键字段...
};
int get_task_struct_info(struct task_info *info);
```

2. 内核侧实现：

- 在 vDSO 镜像 (arch/x86/entry/vdso/vdso.lds.S) 添加新符号 `__vdso_get_task_info`
- 实现函数填充 `current` 宏指向的 `task_struct` 信息
- 用户态访问约束：所有指针字段标记为 `__user` 属性

测试方案

- 对比从 `/proc/self/stat` 获取的 `pid` 与 vDSO 返回值
- 测试多次调用 `fork()` 的接口检查稳定性

用户态调用 vDSO 函数的方式：假如你的 vDSO 函数名为 `_vdso_my_func`，通过以下代码可以获得函数指针

```
void *handle = dlopen("linux-vdso.so.1", RTLD_LAZY);
void *sym = dlsym(handle, "_vdso_my_func");
```

注意事项

- vDSO 是运行在用户态的共享库，没法直接调用内核函数或访问内核数据。你需要一些方式把内核的信息传递到用户态，你可以参考 `vvar` 的实现（开一段 `vma` 用于映射信息）
- 你可能需要关注 `arch/x86/entry/vdso/vma.c`

4.3 无需中断的系统调用

通过 vDSO，我们可以实现无需特权态切换对只读数据的访问。那么对于需要请求内核服务的系统调用，是否也可以避开特权态切换呢？答案是肯定的，FlexSC [3] 给出了对应的解决方案。这一解决方案可以简化为用户态程序和操作系统内核存在共享页面，用户态程序将系统调用需要的参数写进这个共享页面，然后操作系统内核从这个共享页面中读取数据并且将结果写回该页面实现基于共享内存的系统调用。

目标：在 Linux 内核中实现无特权态切换的系统调用服务。

提示：用户态线程比较容易实现请求后等待结果的自旋，需要确保内核处理的线程始终在线。

指标与评测要求

- 吞吐和延迟要求：性能应当可以提升至少 20%。
- 兼容性：需通过 Linux Test Project (LTP) 全部系统调用相关测试用例

测试方案

- 吞吐：使用 Apache Bench 进行百万次 getpid() 调用测试
- 正确性：运行所有 syscall/目录下的 LTP 测试用例

第5章 内存管理

在计算机系统（1）中，我们在虚拟存储器部分引出了交换空间的概念。管理内存是硬件和软件协同的结果。在 CPU 启动后，内存的地址空间完整提供给第一个启动的软件，也就是操作系统。随后操作系统在创建每一个进程时分配页表进行页映射，从而提供每个进程完整虚拟地址空间的抽象。在这个专题的实践中，我们的目标是体验操作系统中不同的内存类型和对应管理内存的方式。

5.1 页表和文件页

目标：读取页表，并且手动修改页表页的映射实现共享（通过 `mmap` 调整映射，通过修改内核读取页表）；设置文件页实现读写落在文件的指定区域中。具体而言，你需要完成以下两个目标：

1. 通过调用 `mmap` 对一个 virtual memory 设定新映射；
2. 通过调用 `mmap` 映射文件后以内存读写的方式访问修改文件（单线程），助教认为你应该需要以下系统调用：`open`、`stat`、`mmap`、`munmap`，助教认为你可能需要以下系统调用：`msync`。

指标与评测要求

文件读写的速度需要能够撑满磁盘带宽。

测试方案

1. 页表项有效性验证
 - 测试原始的虚拟内存存在重新 `mmap` 之后物理地址不同。
2. 内存-文件同步测试
 - 构造 3 组不同测试文件（空文件/4KB 小文件/1MB 大数据文件），每种场景测试读后写、跨页写、追加写操作
 - 通过 `hexdump` 工具进行二进制比对验证

5.2 内存文件系统

Linux 中提供了内存文件系统（RAMfs）的支持，根据 MMU 是否使用可以分为有 MMU 版本和 NoMMU 版本，分别位于 `fs/ramfs/file-mmu.c` 和 `fs/ramfs/file-nommu.c`。RAMfs 无法持久化，在系统崩溃时文件会全部丢失。现在你需要为 RAMfs 提供简单的持久化支持，当某个文件被 `flush` 到 RAMfs 时，该文件同步刷到一个预先设置的同步目录中以持久化该文件。文件系统需要有简单的容错机制，无需保证断电的数据写入，但是必须保证写入的文件原子性操作。（内容可以不完整，但是不能出现 orphan inode）目标：增加 `flush` 下的文件持久化。

接口设计补充：你可以使用 FUSE 在用户态支持，也可以在内核态增加接口。一个可能的接口样式如下：

```
int ramfs_bind(char *sync_dir); // 绑定后端写入的文件
int ramfs_file_flush(struct file *file); // 同步触发接口
```

测试方案

1. 单文件持久化正确性
 - 构造读写请求在以下时机触发 `flush`：立即写入、页缓存（Page Cache）满时写入、定时器触发写入，要求不崩溃

- 验证文件内容和写入的一致性
2. 多线程顺序一致性
- 创建 10 个生产者线程进行高频写操作（含时间戳标记），创建 3 个消费者线程随机触发 flush。验证最终持久化文件中的操作顺序满足 Lamport happens-before 关系。
 - 交错执行 append 和 truncate 操作的临界情况
3. 异常场景测试
- 在 flush 过程中注入断电模拟（通过 sysrq 触发 kernel panic），恢复后验证文件系统的完整性约束

5.3 内存压力导向的内存管理

MacOS 给出内存压力的概念，相比于 UNIX 的 swap 大小，它被认为是更好的表达虚拟内存健康程度的标志。

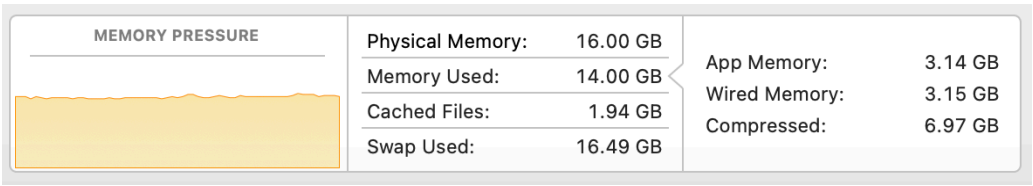


图 5.1: MacOS 中的内存压力

- 目标：该实践任务包含两个子任务点。
1. 利用 Linux 的 Wired Memory（即 Active Memory）和压缩内存给出一个 Linux 下的内核内存压力指示器。
 2. 在用户态划出一块内存空间，实现一个简单的基于内存压力的交换策略。
- 提示：区分 wired memory 和 inactive memory 的区别。
- 测试：测试内存压力和感知准确度的对比（演示），测试交换策略。

第 6 章 文件系统

文件是 UNIX 设计哲学中的一个重要组成部分，一切皆文件的抽象使得用户调整系统的参数也变得更加简单。因此，如何实现一个既能管理系统又能管理日常文件的文件系统便显得更加重要。Linux 通过一套统一的 VFS（虚拟文件系统）抽象达成这一目标。在本实践部分，你需要对 VFS 进行修改并且尝试使用文件系统。

6.1 inode 和扩展属性管理

目标：读取设置 inode 的基本信息，引入新的文件扩展属性 (Extended file attributes)。你需要实现一个 .c 的文件直接调用系统调用调整文件扩展属性。

测试：测试程序会先调用你的程序进行文件扩展属性的读写，随后和标准的文件扩展属性进行比对。你不需要在这个作业中修改内核。

6.2 用户态文件系统

由于文件系统在 Linux 中被编译为内核对象 (kernel object)，文件系统崩溃会传导到内核造成内核不可用。此外，内核对象的灵活性较差。Linux 给出了 FUSE 的服务，允许在用户态实现一个文件系统。

目标：使用 FUSE 实现 GPT 服务。声明一个 GPTfs，其中的目录为每一个对话 session，对话 session 文件夹下的 input 为本轮用户输入的 prompt，output 文件为 GPT 输出的结果。每一轮对话都是单轮对话，无需考虑记录上下文。具体而言，你需要实现：

- 实现一个名为 GPTfs 的 FUSE 文件系统。
- 文件系统的根目录下，每次创建一个新目录即表示开启一个新的对话会话 (session)。
- 在每个对话 session 目录下，包含以下文件：
 1. input：用于存放本轮用户输入的 prompt。
 2. output：用于存放 GPT 生成的回复结果。
 3. error：用于存放网络连接错误等其他错误内容。
- 对 input 文件的写入操作应触发 GPT 处理，生成回复内容并写入对应的 output 文件。
- 每一轮对话都是单轮对话，无需记录上下文。

测试：测试程序会写入 input 文件，随后你应当调用 GPT 服务输出，在这里测试程序会等待 5 秒后检查 error 和 output。

6.3 用户空间下的内存磁盘

随着硬件的发展，内存的价格已经显著下降。内存的访问速度非常快，可以作为一部分反复读写的文件的临时存储。在这一个任务中，你需要从用户态划出一块区域用于磁盘存储。

目标：使用 FUSE 实现 RAMfs 功能，要求读写碎片尽可能少并且支持硬链接。

6.3.1 接口要求

- 实现 FUSE 文件系统需要的回调函数，包括但不限于：
 1. getattr：获取文件或目录的属性。
 2. readdir：读取目录内容。
 3. mknod：创建文件。
 4. mkdir：创建目录。

5. `unlink`: 删除文件。
 6. `rmdir`: 删除目录。
 7. `rename`: 重命名文件或目录。
 8. `link`: 创建硬链接。
 9. `open`、`read`、`write`: 文件的打开、读取和写入操作。
- 设计高效的内存数据结构，管理文件系统的元数据和文件内容。
 - 实现对硬链接的支持，正确更新 `inode` 的引用计数，确保文件在所有链接被删除后才释放内存。
 - 处理并发的读写请求，确保数据的一致性和线程安全。

6.3.2 测试

- 基本的文件和目录操作测试：创建、读取、写入、删除、重命名。
- 硬链接的创建和删除（`inode` 引用计数的正确性以及文件内容的共享）。
- 多线程高并发的读写操作。
- 检查内存使用情况，评估内存泄露和文件碎片。

6.3.3 提示

- 可以参考内核中 `RAMfs` 的设计思路，但不得直接复制代码。
- 注意内存的分配和释放，确保每次分配的内存都有对应的释放操作。
- 使用锁或者其他同步机制来处理并发访问。
- 充分测试各种边界条件，如大文件、小文件、深层目录结构等。

第7章 网络与外部设备

7.1 tcpdump 和 socket 管理

Linux 内核提供了抓包 (capture packets) 等服务作为内核网络套件的一部分。本节的目标包括对 tcpdump 进行定制化修改，以及对内核 Socket 的管理进行改进，从而实现更高的安全性和基于线程级的公平性 (per-thread socket fairness)。

7.1.1 功能目标

1. **定制化 tcpdump 抓包：**在 tcpdump 基础上进行修改，允许用户在不依赖 tcpdump 现有参数的情况下，自行传入过滤规则或者其他控制参数，以满足特定的抓包需求。
2. **Socket 公平性管理：**在内核中为每个线程提供独立的 Socket 资源上限或限流控制 (fairness)，确保多线程在执行网络操作时不会发生资源独占或过度使用的现象。

7.1.1.1 标准函数声明示例

以下仅给出示例，用于说明如何编写对应接口的函数声明，具体细节可根据实际需求进行扩展。

```
/**
 * @brief 使用自定义过滤规则对网络数据进行抓包
 * @param iface 需要进行抓包的网络接口名称
 * @param custom_filter 用户自定义的过滤表达式
 * @param buffer 存储抓取到的数据缓冲区
 * @param buffer_size 存储抓包数据的缓冲区大小
 * @return 成功时返回0，失败返回非0错误码
 */
int custom_tcpdump_capture(const char* iface,
                           const char* custom_filter,
                           void* buffer,
                           size_t buffer_size);
```

Listing 7.1: 示例：自定义 tcpdump 抓包接口

```
/**
 * @brief 为特定线程配置Socket级别的公平管理策略
 * @param thread_id 需要配置的线程ID
 * @param max_socket_allowed 该线程允许打开的最大Socket数
 * @param priority_level 该线程的优先级（影响Socket分配策略）
 * @return 成功时返回0，失败返回非0错误码
 */
int configure_socket_fairness(pthread_t thread_id,
                              int max_socket_allowed,
                              int priority_level);
```

Listing 7.2: 示例：线程级别 Socket 公平性管理接口

7.1.2 tcpdump 定制化抓包

tcpdump 库基于 linux 内核的 libpcap 库实现，提供了对网络数据包的捕获和分析功能。我们可以利用 ‘<pcap.h>’ 中的函数来实现这一功能。具体而言，我们可以通过以下步骤来完成定制化抓包：

1. 使用 ‘pcap_open_live’ 函数打开对应的网络设备。
2. 使用 ‘pcap_compile’ 函数编译用户自定义的过滤规则。
3. 使用 ‘pcap_setfilter’ 应用过滤规则。
4. 使用 ‘pcap_loop’ 函数开始进行抓包，并且进行进一步地处理。

7.1.3 测试

- **定制抓包正确性：**验证是否确实只抓取了满足自定义过滤规则的数据。
- **多 Socket 公平性：**测试多线程同时创建并使用 Socket 时，是否能按照规划的最大数目和优先级进行分配，并观察线程间资源竞争的情况。
- **性能指标：**在系统负载较高时统计 p99 延迟（多 Socket 读写操作）。

7.2 高速通信库：NCCL

由于单节点计算能力的提升，分布式计算对通信带宽和延迟提出了极高要求。传统以太网通信常常无法满足此种高吞吐需求。NCCL (NVIDIA Collective Communications Library) 提供了 GPU 间高速通信接口，可广泛应用于深度学习训练场景。下文给出了相关参考文档以及源代码仓库：

- [Nvidia NCCL examples](#)
- [Nvidia NCCL GitHub repo](#)

7.2.1 功能目标

1. 在多 GPU 环境下，使用 NCCL 实现简单的信息传输（如广播、All-Reduce 等），验证数据传输的正确性。
2. 与以太网通信方式进行带宽和延迟对比，观察速度提升和正确性差异。

7.2.1.1 标准函数声明示例

```
/**
 * @brief 使用NCCL执行多GPU数据广播操作
 * @param data      需要广播的数据缓冲区指针
 * @param count     数据元素的个数
 * @param root      广播发送方的GPU ID
 * @param comm      NCCL通信器
 * @return ncclSuccess表示成功，其他错误码请参照NCCL官方文档
 */
ncclResult_t nccl_broadcast_data(void* data,
                                size_t count,
                                int root,
                                ncclComm_t comm);
```

Listing 7.3: 示例：NCCL 通信例程接口

7.2.2 测试项与测试方式

- **正确性测试：**不同 GPU 间进行广播、聚合 (All-Reduce) 后，结果是否一致，误差是否在可接受范围内（通常为浮点运算误差）。
- **性能测量：**分别采用 NCCL 和以太网 (TCP/IP) 进行跨节点通信，记录带宽、吞吐量、延迟等指标，对比二者差异。

7.2.3 更多细节

具体而言，nccl 库支持的是 GPU 之间的通信。所以具体而言，在当前的接口下，我们总共需要这样几步来完成工作：

1. 为当前的 GPU 创建对应的 cuda stream。
2. 在 gpu 上分配对应的内存空间。
3. 把主存中的数据拷贝到 GPU 上。
4. 调用 nccl 库进行广播。
5. cuda 间进行同步，并且释放 gpu 上空间。

在当前的接口下，这样应该足够完成工作了。

7.3 数据平面开发套件：DPDK

DPDK(Data Plane Development Kit) 是一套在用户态加速网络收发的开发工具包。它将网络协议栈从内核态卸载到用户态中运行，从而极大减少特权级切换 (toggle between privileged levels) 并提升网络吞吐量。本节在有限的复杂度下，演示如何利用 DPDK 实现多进程带宽平衡的 QoS 管理。

7.3.1 功能目标

1. 在用户态基于 DPDK 实现最简化的网络收发功能。
2. 设计一个 QoS 管理策略，使得多个进程共享网络带宽时，能根据设定的策略进行带宽分配并保证平衡。

7.3.1.1 标准函数声明示例

```
/**
 * @brief 在DPDK环境下为队列配置带宽限速
 * @param queue_id    DPDK端口队列编号
 * @param rate_limit  限速值（单位：Mbps）
 * @return 0表示成功，非0表示发生错误
 */
int dpdk_qos_configure_rate_limit(uint16_t queue_id,
                                  uint32_t rate_limit);

/**
 * @brief 发送数据包入口函数
 * @param buffer      需要发送的包数据指针
 * @param len         包长度
 * @param queue_id    发送使用的DPDK端口队列编号
 * @return 发送成功的数据包数，出现错误返回负值
 */
int dpdk_send_packet(const uint8_t* buffer,
                     size_t len,
                     uint16_t queue_id);
```

Listing 7.4: 示例：DPDK QoS 管理接口

7.3.2 测试项与测试方式

7.3.2.1 测试项

- **基本发送接收功能：**在用户态能够正确收包、发包，确保功能可用。

- **多进程带宽平衡：**当多个进程请求同时发送数据时，观察各进程的实际带宽分配是否符合设定的 QoS 管理策略。
- **性能指标：**记录最大吞吐量、p99 延迟，比较在无 QoS 管理与有 QoS 管理时的性能差异。

7.3.2.2 测试方法

1. **功能正确性：**设置单进程收发场景，发送固定大小的数据包，检查收发统计信息。
2. **多进程带宽平衡：**并行启动多个进程，各自不断发送数据包。使用 `dppk_qos_configure_rate_limit` 函数为不同队列设置不同的速率限制（不同版本可能略有不同，可以使用自己期望的版本并且告知助教后进行调整），测试带宽使用情况是否符合预期配额。
3. **压力测试：**持续提高数据发送速率或缩短发包间隔，检测稳定性、丢包率和延迟情况，测试不同 QoS 设置的效果。

第8章 综合应用

当你来到这一章节时，你应该对 Linux 内核的主要组成部分有了清晰的了解，对每个部分的修改效果应该也熟悉了。在这个基础上，我们进一步探讨我们的综合应用。

8.1 使用 RDMA 的远程内存 Swap

在这个项目中，你需要实现一个简单的使用远程内存的交换分区（swap）。具体而言，当缺页异常发生时，你需要使用 RDMA 访问远程内存将页面换到本地。如果远程页面不存在，你需要寻找本地交换文件中寻找这个目标页面。在两者都不存在的情况下，应当给出一个内存错误（如同访问非法地址的错误）。

8.1.1 指标

项目形态。 你的项目最终的形态应当符合以下指标：

1. 将交换分区的内存页面通过 RDMA 存储到远程内存上。
2. 设定一个可调整的比例决定远程内存和本地内存的比例。
3. 对应用无感。

基础评测指标。 项目的基础评测指标如下：

1. 使用 memcached [2] 存储一系列数据（YCSB [1]）在使用 50% 远程内存时在 50% 能够产生一个显著的时间差。
2. 使用 RDMA 的交换时间需要比磁盘交换更短。

对于基础评测指标，一个简单的示意图如图 8.1 所示。

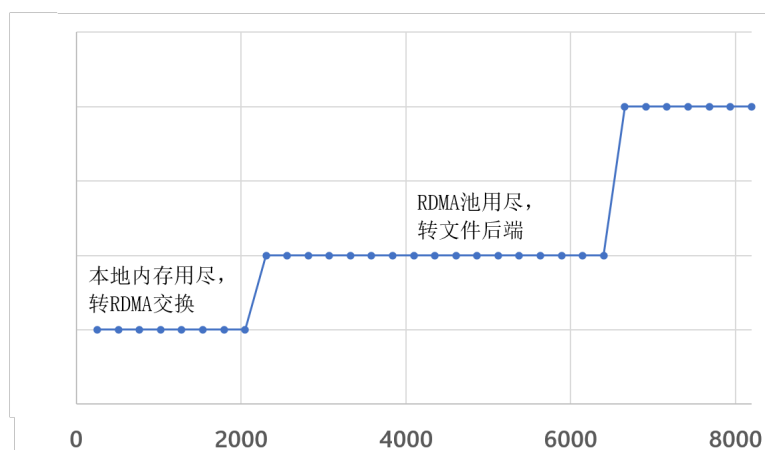


图 8.1: RDMA 和文件混合交换分区下的内存访问量同时间的示意图。图中假设本地内存为 2048MB，RDMA 内存约 4096MB。相对时间仅为示意图，不代表真实场景数据。

进阶指标。 在基础评测的指标上，结合内存访问的特点，实现页预取和交换，目标发生尽可能少的缺页异常。

8.1.2 快速上手和实现重点提示

本项目的核心包括两个部分，内存管理和 RDMA 的访问。对于内存管理部分，需要修改原有系统的 Page Fault 处理流程。RDMA 部分则需要维护 RDMA 连接确保发生缺页异常时向远端设备发起 RDMA 读写请求。

内存管理。 缺页异常发生后，操作系统需要确认这个页位于什么地方（是在内存中但是页表没有映射？还是这个页已经被交换到磁盘设备？或是这个页根本不存在？）。传统的顺序流程可以在体系结构下的 `mm/fault.c` 中找到相关的代码。加入 RDMA 后，需要保证 RDMA 部分不能被换出，并且加入对 RDMA 内存的搜索。

RDMA 管理。 RDMA 的通信与不同的以太网通信不同。为了保证传输的速度，RDMA 没有使用套接字 (Socket) 抽象。取而代之的是两大类 Verbs (Memory Verbs 和 Message Verbs) 这两大类的 Verbs 具体包含：Read (远程主机读取部分内存)、Write (数据写到远端主机)、Atomic (原子操作)；Send (把数据发送到远程 QP 的接收队列里)、Receive (接收主机被告知接收到数据缓冲)。具体的管理可以参考以下链接：(中文) <https://zhuanlan.zhihu.com/p/649468433> (英语官方原版) <https://academy.nvidia.com/en/course/rdma-programming-intro/?cm=446>

特殊点提示。

- 多线程的访问安全性：多个进程同时发生缺页异常在远端的处理过程；
- RDMA 内存的管理：减少远端内存中的内存碎片。

8.2 用户程序嵌入内核执行

用户程序执行时需要通过系统调用 (syscall) 调用特权系统服务，每一次系统调用都有上下文切换的开销，从而使得密集调用系统调用的程序产生额外的开销。一种简单的优化方法是将含有系统调用的程序嵌入内核执行，从而避免上下文切换。这一更改实际上给内核打了很多的“洞”，忽略了内核的权限隔离，因此是不安全的。我们深入之后会发现，这一想法的核心其实是一部分上下文权限隔离对程序而言过强了，程序没有用到对应的功能因此也不需要对应的隔离。在这个基础上，我们可以通过为一些简单程序设计内核中隔离他们程序的方案。具体而言，通过修改 fork 的过程将程序直接嵌入内核空间，增加一些特殊的机制（你自定义的机制，可以是页表、MPK 等）实现该段代码在内核中仅能访问特定内存（相当于自己的地址空间）。假设程序不是恶意的，因此不需要你考虑控制流完整性，但是需要注意代码的映射区域。同时假设在这项作业中程序已经被编译为位置无关代码。

8.2.1 指标

项目形态。 你的项目最终的形态应当符合以下指标：

1. 将一段程序嵌入到内核态执行，程序本身无需修改。
2. 程序仅能访问自己的内存区间，而不能访问内核的其他数据结构。

基础评测指标。 项目的基础评测指标如下：

1. 一段仅包含一系列系统调用的程序可以进入内核实现吞吐的显著提升。
2. 程序访问非法内存应当退出当前程序的执行，给出错误信息（可以在内核日志）并且不影响当前内核的执行。
3. 支持多线程程序的执行，不影响多线程程序的可扩展性。

进阶指标。 在基础评测的指标上，支持带有多个用户库文件的用户程序进入内核执行。

8.2.2 快速上手和实现重点提示

本项目的核心包括两个部分，代码启动阶段 (fork) 和执行阶段 (exec)。

启动阶段 启动阶段用户程序并没有执行，核心需要考虑的部分是如何映射用户段的代码到内核空间，以及如何设置权限隔离的初始状态。此外，需要在启动阶段拦截系统调用，确保系统调用转换为内核态的函数跳转。

执行阶段 执行阶段最核心的是在遇到错误的时候如何修正错误的状态，从而避免嵌入内核的程序崩溃内核。

特殊点提示。

- 多线程执行：与内核线程的绑定执行关系；
- 内核状态的保护：防止单个程序崩溃整个内核。

8.3 内核空间下的内存磁盘

随着硬件的发展，内存的价格已经显著下降。内存的访问速度非常快，可以作为一部分反复读写的文件的临时存储。这一部分是用户空间下的内存磁盘的进一步延伸。

8.3.1 指标

项目形态。 你的项目最终的形态应当符合以下指标：

1. 将一段内存空间作为磁盘空间映射给内核。
2. 映射的磁盘空间可以进行文件读写和文件夹创建。
3. 注册为一个磁盘设备而非文件系统。

基础评测指标。 项目的基础评测指标如下：

1. 使用 `dd` 一次性创建一个文件的开销足够小（需要接近内存的读写速度）。
2. 多次创建删除文件之后实际存储容量始终保持一致。

进阶指标。 在基础评测的指标上，减小文件元数据的开销大小。

8.3.2 快速上手和实现重点提示

本项目的核心是内存空间的映射和分区管理。与映射在用户空间不同，内核空间的映射需要保证用于虚拟磁盘的数据不能破坏内核数据的正确性。此外，用户空间可以通过 `fuse` 实现用户态的分区访问。在内核中，该模块需要实现为内核模块，因此需要符合内核的一系列数据接口。

特殊点提示。

- 多线程执行：多线程读写保证数据的一致性。
- 文件碎片：文件的碎片应该尽可能少。

8.4 机密虚拟机 Trapless 的宿主协同内存管理

机密虚拟机的一个核心是内存需要加密以防止宿主可以获取到虚拟机内存造成数据泄露，然而这样的安全措施也造成了宿主机无法感知虚拟机内的内存情况，内存分配无法最优化。一种方式是虚拟机内核主动向宿主机的虚拟机管理器发起调用引导宿主机按照一定的偏好管理内存。这种方式需要下陷到宿主机的虚拟机管理器，有较大的切换开销。请设计一种机制，以不下陷的方式引导宿主机的按照一定偏好管理内存。

8.4.1 指标

项目形态。 你的项目最终的形态应当符合以下指标：

1. 不修改虚拟机内应用程序，允许修改虚拟机管理器和虚拟机内的内核。
2. 指示内存偏好时不发生下陷。

基础评测指标。 项目的基础评测指标如下：

1. 使用内存密集型任务在机密虚拟机内部设置特定内存位置后计算速度和用户态的读写速度一致。

8.4.2 快速上手和实现重点提示

本项目的指标较少，因为核心代码需要较大的修改，包括三个部分：宿主机的内核、虚拟机的内核、宿主机的虚拟机管理器。宿主机内核需要设定特殊的接口管理加密的内存页面。虚拟机内核需要修改实现无下陷与宿主机的虚拟机管理器进行交互。宿主机的虚拟机内存管理器需要增加接收虚拟机内核的特殊内存偏好参数并且传递给宿主机。

特殊点提示。

- 虚拟机下陷的处理。
- 共享内存页的管理。
- 宿主机的内存管理。

参考文献

- [1] Brian F Cooper et al. “Benchmarking cloud serving systems with YCSB”. In: *Proceedings of the 1st ACM symposium on Cloud computing*. 2010, pp. 143–154.
- [2] Brad Fitzpatrick. “Distributed caching with memcached”. In: *Linux journal* 2004.124 (2004), p. 5.
- [3] Livio Soares and Michael Stumm. “{FlexSC}: Flexible system call scheduling with {Exception-Less} system calls”. In: *9th USENIX Symposium on Operating Systems Design and Implementation (OSDI 10)*. 2010.

附录 A 细节说明

A.1 环境配置

A.1.1 EDK2

以下是一个编译 EDK2 的 OVMF 和相关 efi 的示例脚本：

```
#!/bin/bash

# 获取脚本所在目录的绝对路径
SCRIPT_DIR="$(cd "$(dirname "${BASH_SOURCE[0]}")"&&pwd)"
# 获取项目根目录（假设脚本在 tools/scripts 目录下）
PROJECT_ROOT="$(cd "${SCRIPT_DIR}/../.."&&pwd)"

cd ${PROJECT_ROOT}/edk2

export WORKSPACE=${PROJECT_ROOT}/edk2
export EDK_TOOLS_PATH=${PROJECT_ROOT}/edk2/BaseTools
export CONF_PATH=${PROJECT_ROOT}/tools/config/edk2 # 默认是 edk2/Conf，为了优雅这里把它单独提出来

source edksetup.sh

make -C BaseTools -j$(nproc)

build

build -p ShellPkg/ShellPkg.dsc

build -p YourPkg/YourPkg.dsc
```

以下是一个一键启动 EDK2 UEFI 环境的示例脚本：

```
#!/bin/bash

# 获取脚本所在目录的绝对路径
SCRIPT_DIR="$(cd "$(dirname "${BASH_SOURCE[0]}")"&&pwd)"
# 获取项目根目录（假设脚本在 tools/scripts 目录下）
PROJECT_ROOT="$(cd "${SCRIPT_DIR}/../.."&&pwd)"

# 定义资源文件路径
OVMF_CODE="${PROJECT_ROOT}/edk2/Build/Ovmf3264/DEBUG_GCC5/FV/OVMF_CODE.fd"
OVMF_VARS="${PROJECT_ROOT}/edk2/Build/Ovmf3264/DEBUG_GCC5/FV/OVMF_VARS.fd"
# SHELL_EFI="${PROJECT_ROOT}/edk2/Build/Shell/DEBUG_GCC5/X64/ShellPkg/Application/Shell/EA4BB293-2D7F-4456-A681-1F22F42CD0BC/DEBUG/Shell.efi"
RAW_ACPIVIEW_EFI="${PROJECT_ROOT}/edk2/Build/Shell/DEBUG_GCC5/X64/ShellPkg/Application/AcpiViewApp/AcpiViewApp/DEBUG/AcpiViewApp.efi"
HELLO_WORLD_EFI="${PROJECT_ROOT}/edk2/Build/YourPkg/DEBUG_GCC5/X64/YourPkg/Application/HelloWorld/HelloWorld/DEBUG/HelloWorld.efi"
HALLO_WORD_EFI="${PROJECT_ROOT}/edk2/Build/YourPkg/DEBUG_GCC5/X64/YourPkg/Application/HalloWord/HalloWord/DEBUG/HalloWord.efi"
MY_ACPIVIEW_EFI="${PROJECT_ROOT}/edk2/Build/YourPkg/DEBUG_GCC5/X64/YourPkg/Application/AcpiView/AcpiView/DEBUG/AcpiView.efi"

# 检查上述文件是否存在
```

```

RESOURCE_LIST=("$OVMF_CODE" "$OVMF_VARS" "$RAW_ACPIVIEW_EFI" "$HELLO_WORLD_EFI" "$HALLO_WORD_EFI" "
$MY_ACPIVIEW_EFI")

for RESOURCE in "${RESOURCE_LIST[@]}; do
    echo "检查文件:␣$RESOURCE"
    if [ ! -f "$RESOURCE" ]; then
        echo "错误: $RESOURCE␣不存在, 请确认编译路径是否正确"
        exit 1
    fi
done
# 创建运行目录 (如果不存在)
PLAYGROUND_DIR="${PROJECT_ROOT}/playground"
rm -rf "$PLAYGROUND_DIR"
mkdir -p "$PLAYGROUND_DIR"

# 复制OVMF变量文件 (避免修改原始文件)
cp "$OVMF_VARS" "${PLAYGROUND_DIR}/OVMF_VARS.fd"

mkdir -p "$PLAYGROUND_DIR/uefi"
# cp "$SHELL_EFI" "${PLAYGROUND_DIR}/uefi/Origin_Shell.efi"
cp "$RAW_ACPIVIEW_EFI" "${PLAYGROUND_DIR}/uefi/O_AcpiViewApp.efi"
cp "$MY_ACPIVIEW_EFI" "${PLAYGROUND_DIR}/uefi/My_AcpiView.efi"
cp "$HELLO_WORLD_EFI" "${PLAYGROUND_DIR}/uefi/HelloWorld.efi"
cp "$HALLO_WORD_EFI" "${PLAYGROUND_DIR}/uefi/HalloWord.efi"

# 暂停
# read -p "按任意键继续..."

# 启动QEMU进入UEFI shell
qemu-system-x86_64 \
    -machine q35,accel=kvm \
    -m 8G \
    -smp 4 \
    -drive if=pflash,format=raw,unit=0,file="$OVMF_CODE",readonly=on \
    -drive if=pflash,format=raw,unit=1,file="${PLAYGROUND_DIR}/OVMF_VARS.fd" \
    -drive file=fat:rw:"${PLAYGROUND_DIR}/uefi",format=raw,if=ide,index=0 \
    -nographic \
    -no-reboot \
    -serial mon:stdio

```

A.1.2 添加动态链接库

以下是一个将程序所依赖的动态链接库添加到 initramfs 里的脚本:

```

#!/bin/bash

# Usage: ./add_binary.sh /path/to/binary
# E.g.: ./add_binary.sh /usr/bin/acpidump

if [ -z "$1" ]; then
    echo "Usage:␣$0␣/path/to/binary"
    exit 1
fi

# This script assumes that the project root is the parent directory of the script.
# You may change the path definition as needed.

```

```

script_dir=$(dirname "$(realpath "$0")")
project_root=$(realpath "$script_dir/..")

BINARY="$1"
BINARY_NAME=$(basename "$BINARY")
INITRAM_DIR="$project_root/kvm/busybox-1.36.1/_install"

# Make sure we're not missing any directory
mkdir -p "$INITRAM_DIR/bin"
mkdir -p "$INITRAM_DIR/usr/bin"
mkdir -p "$INITRAM_DIR/lib"
mkdir -p "$INITRAM_DIR/lib64"
mkdir -p "$INITRAM_DIR/lib/x86_64-linux-gnu"

# Copy the binary
if [[ "$BINARY" == /usr/bin/* ]]; then
    cp "$BINARY" "$INITRAM_DIR/usr/bin/"
    chmod +x "$INITRAM_DIR/usr/bin/$BINARY_NAME"
else
    cp "$BINARY" "$INITRAM_DIR/bin/"
    chmod +x "$INITRAM_DIR/bin/$BINARY_NAME"
fi

# Get and copy all dependencies
ldd "$BINARY" | grep "=>" | awk '{print $3}' | while read -r LIB; do
    if [ -n "$LIB" ]; then
        # Create target directory
        TARGET_DIR=$(dirname "$LIB")
        mkdir -p "$INITRAM_DIR$TARGET_DIR"

        # Copy library
        cp "$LIB" "$INITRAM_DIR$TARGET_DIR/"
    fi
done

# Don't forget the dynamic linker (ld-linux)
LINKER=$(ldd "$BINARY" | grep "ld-linux" | awk '{print $1}')
if [ -n "$LINKER" ]; then
    LINKER_DIR=$(dirname "$LINKER")
    mkdir -p "$INITRAM_DIR$LINKER_DIR"
    cp "$LINKER" "$INITRAM_DIR$LINKER_DIR/"
fi

echo "Added $BINARY_NAME and its dependencies to initramfs"

```